

s of significant subsidiaries or assets;

- major new litigation or regulatory inquiries or developments in existing litigation or inquiries;
- significant cybersecurity incidents or data breaches;
- significant developments in borrowings, or financings or capital investments;
- significant changes in financial condition or asset value or liquidity issues;
- changes in GitLab's Board of Directors, E-group, or senior management;
- significant changes in corporate strategy;
- changes in accounting methods and write-offs; and
- stock offerings, stock splits or changes in dividend policy.

The list of examples is not a comprehensive list of what could be considered material information. Determining what may be material information will depend upon the facts and circumstances in each particular situation.

Helpful Legal References

Product disclaimer templates

Frequently Asked Questions

Do product PIs (performance indicators) need to be marked private?

Yes. Whether in epics, issues or handbook pages, PIs should be internal-only and follow the SAFE framework.

What are some practical examples of things that are SAFE?

Examples of what's safe to share and those that need to remain internal-only:

Situation	Classification	Why
-----------	----------------	-----

---	---	---
-----	-----	-----

Publicly discussing a group's MAU in an issue	Not SAFE	We should avoid publicly disclosing MAU numbers
---	----------	---

Publicly discussing new/potential product performance indicators	SAFE	We can publicly discuss PIs as long as no actual data (numbers/values) is referenced or exposed
--	------	---

Requesting a review from legal

If you need a review from Legal, please ping in the SAFE channel in Slack.

What artifacts/actions can generally be published with a disclaimer?

See the table above. If you are recording a video that requires a disclaimer but are not using a slidedeck, you may alternatively verbalize the disclaimer at the beginning of the video to avoid extra post-production work.

What limitations do we have on sharing specific instrumentation data in public issues?

None, so long as the instrumentation data does not contain sensitive, financial, or internal-

only data.

What third-party materials can be included in publicly-available artifacts?

Guidelines covering the use of third-party materials (like images, diagrams, and quotes) are being prepared and will be linked here when ready.

What third-party company and product names and logos can be included in publicly-available artifacts?

The Third-party Trademark Usage Guidelines cover the use of third-party trademarks in GitLab (the product), and set out the process for adding new third-party logos to GitLab.

This advice covers the use of third-party trademarks in the context of comparative advertising.

Should our planning issues be confidential? And if so, should our Kickoff videos be private as well?

In general, keep the issues and videos public. For issues that are of possible interest to investors keep private until publicly disclosed. Don't make public videos that contain earnings and other material information prior to these details being publicly available. Private issues that might find their way to an investor, analyst, or investment institution, are more likely to be in violation.

Should we avoid using \$tag on Social Media posts?

\$cashtags are commonly used to reference anything that is financially related to a company (example). There is nothing wrong with using this symbol to reference or link to any financial information that has already been disclosed and distributed in an authorized public channel. For example, linking to a company's annual or quarterly statements published on their public website.

If the information is not SAFE, do not link to it or share it publicly.

Can you provide a definition for Information that is considered material?

Referencing the SAFE handbook page is a great place to start. If you're still unsure, please reach out to Legal in legal and someone will be glad to assist you.

SECTION: product/product-processes/sensing-mechanisms.md

What are sensing mechanisms?

Our ability to iterate quickly is a measure of our efficiency, but our effectiveness is just as critical. As a product manager you are critical to us not just working correctly, but working on the correct things. You do that by prioritizing appropriately. Your prioritization decisions will be enhanced if you maintain a sufficient understanding of the context in which you make them.

There is no limit to the amount of inputs you can utilize for making prioritization decisions. We've organized these mechanisms into three lists. One for those that primarily sense feedback from users, one that primarily senses feedback from buyers and another that senses internally generated feedback which could represent buyers or users. For new PMs consider these lists as guidance for places to ensure you are plugged in to maintain sufficient context.

In 2020, we conducted a GTM and R&D sensing mechanism survey to get an understanding of how team members valued the sensing mechanisms listed below. We calculated this value by multiplying their responses (1-5) for how efficient they felt a sensing mechanism was by how their responses (1-4) for how effective they felt the sensing mechanism was. We've ranked the below lists by the average value from all respondents.

Sensing Mechanisms by Type

User

- 1. Asking probing questions in customer discovery meetings
- 1. Engaging with our community alongside our Developer Relations team and triaging community generated issues and ideas
- 1. Reviewing top up voted issues
- 1. Engagement directly with customers and via the customer label
- 1. Requesting and analyzing results from UX research.
- 1. A summary of previous user interviews can be found under the User interview project
- 1. All UX Research is being transcribed in Dovetail

Buyer

- 1. Asking probing questions in sales support meetings
- 1. Reviewing Customer Success designated top issues
- 1. Reviewing the most requested issues from customers using the Customer Requested Issues (Product) dashboard and the Customer Issues Prioritization Framework handbook page for guidance.
- 1. Tracking open source projects in the same space as part of your competitive analysis is important as well. You can evaluate these open source options not just for interesting features and ideas, but potentially to integrate them in our product
- 1. Chorus transcriptions of sales calls and demos (how to video - private)
- 1. Reviewing Win/Loss reports
- 1. Learn about customer health data using Gainsight built on salesforce, managed by the Customer Success team
- 1. Attending QBRs and consuming QBR summaries/highlights
- 1. Speaking directly with users via PCSAT responder outreach

Market

- 1. Maintaining competitive and market assessments. Checkout this great video discussing competitive analysis for Product Managers at Product League with GitLab's Orit Golowinski.
- 1. Monitoring and maintaining missing features in your category epics (competitive landscape section)

1. Monitoring and maintaining the direction page for the categories you own
1. Subscribing to your competitor's blogs to be aware of what they are releasing will help you here
1. Reviewing relevant analyst reports
1. Consuming competitive and market content review highlights
1. Reviewing relevant analyst reports
1. Meeting with analysts to confirm direction and vision of section and stages
1. Paying regular attention to competitive wins and losses

Internal

1. Leadership OKRs set the direction for the company
1. Each PM should be having regular conversations with their stage groups stable counterparts to discuss their strategy and plan. Share this discussion with the company via our GitLab Unfiltered YouTube channel. PMs should share their next three milestones, year-long plan, strategy, and relevant OKRs (with status) so everyone can contribute feedback.
1. Dialogue with internal customers to improve the dogfooding potential of your features
1. Quarterly cross-section direction reviews
1. Reviewing Performance Indicators for trend shifts and changes

SECTION: product/product-processes/tiering-guidance-for-features.md

On this page

Tiering strategy

Free is targeted at individual contributor developers. It is a complete DevOps solution and contains capabilities from all ten GitLab stages.

Premium is targeted at Director level buyers and is for teams. The pricing themes for Premium are Faster code reviews, Advanced CI/CD, Enterprise agile planning, Release controls and Self managed reliability. Premium helps teams iterate faster and innovate together.

Ultimate is targeted at Executive level buyers and is for organizations. The pricing themes for Ultimate are Advanced security testing, Security risk mitigation, Compliance, Portfolio management, and Value stream management. Ultimate helps organizations deliver better software faster with enterprise ready planning, security and compliance.

How to make pricing tier decisions

Product Managers are responsible for helping determine and maintain the optimal tier for their features. To accomplish this, Product Managers should leverage the supporting resources below to understand and implement the strategy, philosophy and various components of GitLab's pricing model.

Pricing Tiers

The CEO is the DRI for pricing and tiers. This includes any changes that directly impact how we

charge customers under our licensing model, such as a change to how we handle active users. GitLab leverages a buyer-based open core pricing model. Please review the entirety of the stewardship and pricing model pages before making any determinations as to which tier a given feature should go in.

Determining the tier of a new feature

The likely buyer determines which tier, and "Who cares most about the feature" ultimately determines the likely buyer. Our stewardship principles also help guide whether something belongs in a paid tier. Be sure to consider documenting your rationale for the decision in the issue description, including a reference to our stewardship page when appropriate.

Please indicate the applicable tier for an issue by applying the label associated with the tier (e.g. GitLab Ultimate). Ensure this is defined before feature development begins.

Leverage open source to drive early usage

When early in a category's maturity, it is recommended to start with functionality in the open source version to drive broad awareness and adoption. Early adoption translates into open source contributions, which can help accelerate maturity in a new area of the product.

Have a ladder tiering strategy that leverages all three tiers

Each product area should have a ladder tiering strategy, where incremental value is contained in each tier. This enables us to drive broad usage in Free, and then drive uptiers to Premium and Ultimate as customers increase in their sophistication of using the given product area. When considering what to offer in open source, also think through what additional features can be monetized later. Open source drives popularity of the main feature, but it's important to understand up front how we intend to leverage that usage and exposure to drive incremental ARR later in higher tiers.

Moving features between tiers or other pricing changes

To propose changing a feature tier or making any other change that impacts how we charge customers, please follow the process and template on the pricing page. This ensures collaboration and alignment with key GitLab stakeholders and maintenance of features.yml as SSOT.

Please remember our promise not to make open-source features source-available.

Paid Tier requirements

All Premium, and Ultimate features must:

- Work easily for our customers that self-host GitLab. i.e. Their licenses need not be updated and the new feature is default-on for the instance.
- Work with GitLab.com Premium / Ultimate subscriptions. This means there has to be some way of toggling or using the feature at a namespace level.
- Have documentation.
- Be featured on products and DevOps tools at launch.

Should product managers have any questions when making tier decisions, they should collaborate with their manager, product leadership, or the CEO for clarification. The most up to date reference for pricing DRIs can be found in the feature tier or pricing change template.

Reasons for upgrade

Multiple considerations go into customers' purchase decisions. Here are some various resources product managers can visit to reference various data points for analysis:

- Post purchase surveys (link no longer available)

How to consider impact to revenue

Driving revenue

Product managers should be familiar with and leverage strategies and tactics for their own stage as well as across GitLab's other stages in accordance with GitLab's pricing model. Here are some helpful examples:

- Create stage's use of popularity to drive revenue
- Plan stage's balance between usage and enterprise adoption
- Enablement section's balance of ease of use and scale

Understanding Investment

GitLab currently has three ways of allocating investment across product as detailed in investment types.

Learning opportunities

Pricing adjustments within a buyer-based model can be challenging and sometimes feel counterintuitive. Below are some examples of strategies/tactics that have succeeded and failed that we can learn from:

.

- Move Feature Flags to Core
- With pressure from the market that our SAST features were just open source tools and any customer could just run those themselves for free, we moved SAST analyzers to core. We were able to turn this criticism into a community feature which has increased our community contributions, provided an entry point for all customers to try our security tooling and even create some in-product upgrade paths to Ultimate. Epic, Internal Discussions, Release Post
- The Kubernetes Agent move to core generated great discussions on how competitive pressure and the desire for feedback can challenge the buyer-based tiering

.

- Pricing the MR Diff code quality annotations into Ultimate was something that got some resistance initially since the feature would be used by developers. The feature fits into the Transformation type sale where a team is trying to shift code quality to the left and bring it

to the full code review process.

- Putting the Accessibility MR Widget into Core was about getting some usage for the new category and to learn what additional features customers wanted.

SECTION: product/product-processes/analytics-instrumentation-guide/_index.md

Analytics Instrumentation Overview

At GitLab, we collect product usage data for the purpose of helping us build a better product. Data helps GitLab understand which parts of the product need improvement and which features we should build next. Product usage data also helps our team better understand the reasons why people use GitLab. With this knowledge we are able to make better product decisions.

There are several stages and teams involved to go from collecting data to making it useful for our internal teams and customers.

Stage	Description	DRI	Support Teams
----	-----	---	-----
Privacy Settings	The implementation of our Privacy Policy including data classification, data access, and user settings to control what data is shared with GitLab.	Analytics Instrumentation	Legal, Data
Collection	The data collection tools used across all GitLab applications including GitLab SaaS, GitLab self-managed, CustomerDot, VersionDot, and about.gitlab.com. Our current tooling includes Snowplow, Service Ping, and Google Analytics.	Analytics Instrumentation	Infrastructure
Extraction	The data extraction tools used to extract data from Product, Infrastructure, Enterprise Apps data sources. Our current tooling includes Stitch, Fivetran, and Custom.	Data	
Loading	The data loading tools used to extract data from Product, Infrastructure, Enterprise Apps data sources and to load them into our data warehouse. Our current tooling includes Stitch, Fivetran, and Custom.	Data	
Orchestration	The orchestration of extraction and loading tooling to move data from sources into the Enterprise Data Warehouse. Our current tooling includes Airflow.	Data	
Storage	The Enterprise Data Warehouse (EDW) which is the single source of truth for GitLab's corporate data, performance analytics, and enterprise-wide data such as Key Performance Indicators. Our current EDW is built on Snowflake.	Data	
Transformation	The transformation and modelling of data in the Enterprise Data Warehouse in preparation for data analysis. Our current tooling is dbt and Python scripts.	Data	Analytics Instrumentation
Analysis	The analysis of data in the Enterprise Data Warehouse using a querying and visualization tool. Our current tooling is Tableau.	Data, Product Data Insights	Analytics Instrumentation
Post Launch Instrumentation	Increase product instrumentation across our features to deliver greater product insights. There is a need to retroactively evaluate what features have been instrumented and need instrumentation from past feature launches. Post launch implementation will allow us to gather insights that currently are being missed and to allow our CSM team to assist customers in seeking to understand feature usage + adoption within their organizations	Product, Product Data Insights	Analytics Instrumentation

Editable source file

Quick Links

| Resource | Description |

| ----- | ----- |

| [Getting started with Analytics Instrumentation](#) | The guide covering implementation and usage of Analytics Instrumentation tools |

| [Metrics Dictionary](#) | A SSoT for all collected metrics and events |

| [Privacy Policy](#) | Our privacy policy outlining what data we collect and how we handle it |

| [Product Usage Data Privacy Policy](#) | Our privacy policy outlining product usage data we collect and how we handle it |

| [Analytics Instrumentation Direction](#) | The roadmap for Analytics Instrumentation at GitLab |

| [Analytics Instrumentation Development Process](#) | The development process for the Analytics Instrumentation groups |

2024-05-16: last page update

SECTION: product/product-processes/analytics-instrumentation-guide/service-usage-data-commitment.md

Our Commitment to Individual User Privacy in relation to Service Usage Data

While there are examples of data collection used for malicious intent, data collection and analysis has also allowed companies to improve their product or service, benefiting their end user/consumer. It is in this vein, that GitLab collects usage data about its products. We collect individual usage data in a pseudonymized manner at the namespace level and then use this information to power our product decisions and improve GitLab for you. We may also aggregate all this information to understand broadly how GitLab product is used.

How we use Service Usage Data

On our Product Performance Indicators page, under each graph there is a Lessons Learned callout which summarizes insights and opportunities based on the usage data collected. The improvements made to the product through this process are largely attributable to usage data collection.

As an example, the Package team at GitLab watches the usage metrics related to the count of users who published a package to the Package Registry over time. To be clear, the data they are analyzing is in an aggregate form, no user-identifiable information is analyzed. As a result of digging into the trends they identified this insight:

>From a funnel perspective, we saw significant growth in the packages pulled using a deploy token or by a Guest. Both are signs that the Package Registry is being integrated into our customers production workflows.

Based on this analysis, the team prioritized two bug issues related to deploy tokens. This is an efficient and effective use of usage data which never put in jeopardy a person's identity, nor GitLab's credibility.

This is the data space in which we operate and will continue to operate.

Our Commitment to GitLab Users

Over the past few years GitLab has made commitments to our community around the collection, processing, and use of service usage data. This page summarizes those commitments and provides guidance to team members working on projects that involve the collection of product analytics data from our customers.

1. We will not sell your service usage data.

1. GitLab will only add free JavaScript (as in freedom) to its core product. (source)

1. We pseudonymize personally identifying information about you, the user.

1. We will transparently document the data we collect, how it is used and how it is pseudonymized.

1. We will alert the community when any changes to our service usage data processes and/or policies occur.

1. We will not implement telemetry in our products which sends identifiable usage data to a third-party product analytics service source

Definition of Analytics Data

Analytics Data can be too generic of a term. The list below are the specific types of Product data that is in scope:

- Service Ping - aggregate counts of product usage from a GitLab instance
- Snowplow Events - dynamic events collected from either a client or server.
- Database events - using Database records from gitlab.com to measure and understand usage

Data Pseudonymization

As many other organizations offering digital products, GitLab strives to get better understanding on utility of its offering. In order to build the best DevOps platform for everyone, we try to understand what are the most used areas, which get overlooked, which are in need of improvement, and which we can be proud of.

To get to such understanding, we look to collected service usage data. And as part of collecting usage data, we aim to provide robust privacy protection, and assurance that this data would not be ill-used. With that obligation in mind, we are working to prepare a privacy protection mechanism that would include technical tools and various policies.

Pseudonymization Isn't Perfect

While we will be pseudonymizing personal information for individual users, there are cases where a project or namespace could be identified. There are a few primary examples:

1. We will collect projectID, it can be used to identify the project name via our APIs but this is only true for projects set to Public visibility where you are a member.

1. We will collect namespaceID, it can be used to identify the namespace name (which may be a personal name) via our API you can only return namespace information for namespaces you are a member of.

1. In the case of single user subscribers, product usage activity can be tied back to an actual user (internally) via metadata in our enterprise applications.

Analytics Instrumentation Roadmap

Our pseudonymization process to de-identify personally identifiable data which relies on one-way hashing is was released in milestone 14.4 (October 2021).

A key part of our process is pseudonymizing data at the collection layer, which allows GitLab to resolve any issues without a dependency for upgrading versions on your part.

Now that we have the ability to protect user privacy with the pseudonymization service in place, we have started collecting ProjectID, NamespaceID and pseudonymized UserID. Collecting these identifiers make the aggregated metrics we collect much more revealing. Now, instead of know there were 1000 clicks of some button, we can know things like: "Unidentified User "X" clicked a button, performed an action, then hit an error." This rich user journey will greatly improve GitLab's ability to improve our product for you, our end user.

Next up for our roadmap includes modeling user journeys to better understand the features are users value most and implementing events track in self-managed instances.

Related Content

- 1. GitLab's Privacy Policy
- 1. Updates to Pseudonymizing Service Usage Data blog post: coming soon
- 1. Update on Free Software and Telemetry
- 1. Product Usage Tracking Issue
- 1. GitLab's Analytics Instrumentation Direction

SECTION: product/product-processes/pm-procedures/_index.md

Overview

This section of the handbook is a collection of processes that are required to be followed under certain conditions. For example, if a change is being made or if a request is submitted to leadership for approval.

How this page works

In the spirit of "every can co-create", these procedures can be contributed by any one in the Product Division (or even GitLab!). The custodian of the procedures are Program Management. If you are interested in contributing to this page, please open a merge request and assign to Natalie Pinto, GitLab Handle @natalie.pinto.

Procedures

Making changes to stages, groups, or categories

Documentation on how to make changes to stages, groups, and categories, as well as what approvals are required, can be found in our website handbook page.

As GitLab grows we will need to create new groups, stages, and categories. During this transition period we need to globally optimize, and ensure that important issues are not blocked during the creation of a new group.

There are three common scenarios which may encounter these transition periods:

- Splitting a Stage into multiple Groups
- Changing the Categories in a Stage
- Adding a new Stage
- Creating new Groups
- Moving an existing categories to a different Group

In each of these scenarios, a Group may become temporarily responsible for a future Group's issues and backlog. During this time, the responsible Group should balance the needs of our users and organization, prioritizing across the common backlog.

For every scenario above, we also need to ensure that the Engineering Metrics dashboard is updated to correctly track the new label.

The label change affects both Issues and Merge Requests. The Stage and Group labels power the visualization and metrics in the Quality and GitLab Insights dashboards.

Please create a new issue in the Triage Ops with the template label-change.md.

Creating new Groups

As the group(s) working on a Stage grow in size, a new Group may need to be formed so group sizes remain manageable. For example, today there is a single group for Manage, but new groups were created for control and framework.

To prepare for creating multiple Groups, we should:

1. Update categories.yml and stages.yml, assigning each Group a set of Categories
1. Ensure all issues remain labeled with the Stage name, like devops::manage
1. Ensure all issues also have a group label, like Control or Framework
1. Prior to the new groups being formed, the PM and EM prioritize the shared devops::manage backlog

Once the first PM or EM is hired, the new Groups should be formed:

1. The other PM/EM's will need to continue working across both Groups. For example if a backend EM is hired, the frontend EM and PM will continue to work across both Groups until additional hires are made.
1. EM's and engineers should work together and divide the existing engineering team to staff the new groups, like Control and Framework. Ideally, each group would have at least two backend engineers, but that is not always possible.
1. Update stages.yml and team.yml to reflect the new group members. You may need to add a member to the end of categories.erb.

1. Create a Slack channel for each group, like gcontrol and gframework, and close the previous Slack channel (e.g. gmanage)

As the rest of the EM/PM's are hired, they take over that role for the respective group.

Note: We used to call this "splitting" a group. However, we adjusted it because it's important to emphasize with group members that they are forming new independent groups and should feel free to independently create their new norms and processes.

Splitting a Stage into multiple Groups

As the group(s) working on a Stage grow in size, a new Group may need to be formed so group sizes remain manageable. For example, today there is a single group for Manage, but will be splitting into control and framework groups.

To prepare for splitting a Stage into multiple Groups, we should:

1. Update categories.yml and stages.yml, assigning each Group a set of Categories
1. Ensure all issues remain labeled with the Stage name, like devops::manage
1. Ensure all issues also have a group label, like Control or Framework
1. Create a "Label change" issue in Triage Ops listing affected label to have the change reflected retroactively in Engineering Dashboards.
1. Prior to the new groups being formed, the PM and EM prioritize the shared devops::manage backlog

Once the first PM or EM is hired, the new Group should be formed:

1. The other PM/EM's will need to continue working across both Groups. For example if a backend EM is hired, the frontend EM and PM will continue to work across both Groups until additional hires are made.
1. EM's and engineers should work together and divide the existing engineering team to staff the new groups, like Control and Framework. Ideally, each group would have at least two backend engineers, but that is not always possible.
1. Update stages.yml and team.yml to reflect the current group members. Update categories.erb with the member names, if necessary.
1. Create a Slack channel for each group, like gcontrol and gframework, and close the previous Slack channel (e.g. gmanage)

As the rest of the EM/PM's are hired, they take over that role for the respective group.

Adding a new category to a Stage

The categories within a Stage will change over time, based on GitLab's direction and the market landscape. The groups within a Stage will need to be able to handle these changes, without issues falling in the cracks.

When the categories change, we should:

1. Update categories.yml and stages.yml, ensure all categories are assigned to a Group
1. If two categories are merging, apply the new category label to issues from both of the old

categories

1. If a new category is being added, create a new category label and apply it to relevant issues

1. Create a "Label change" issue in Triage Ops listing affected label to have the change reflected retroactively in Engineering Dashboards.

1. Update category strategy to reflect the new labels and categories

1. Review the handbook and other material which may link to old categories

1. Archive old category labels, this will be done by the Quality Department as part of the "Label change" issue.

Adding a new Stage

When GitLab decides to address additional needs within the single application, a new Stage may need to be created. For example, Software Supply Chain Security may be created to address additional needs beyond what Secure focuses on.

When a new Stage is added, and its Group has yet to be formed, we should:

1. Ensure all issues for the new Stage are assigned with the Stage labels, like `devops::software-supply-chain-security` and Software Supply Chain Security.

1. Create a "Label change" issue in Triage Ops listing affected label to have the change reflected retroactively in Engineering Dashboards.

1. Identify an existing Group, like Secure, which will be initially responsible for the new Stage

1. The existing Group will prioritize across a common backlog of both Stages, in this example `devops::software-supply-chain-security` and `devops::secure`

1. Update `categories.yml` and `stages.yml`, listing the new Stage with the members of the existing responsible Group. Update `categories.erb` with the member name, if necessary.

Once the first PM or EM is hired, a new Group for the Stage should be formed:

1. The other PM/EM's will need to continue working across both groups. For example if a backend EM is hired, the frontend EM and PM will continue to work across both groups until additional hires are made.

1. EM's and engineers should work together to staff the new Group, like govern. Each Group should have at least two backend engineers.

1. Now that the new Group is formed, both Groups can focus on their respective Stages. In this case, Secure on Secure and Software Supply Chain Security on Software Supply Chain Security.

1. Update `stages.yml` to reflect the new Group and its members. Update `categories.erb` with the member name, if necessary.

As the rest of the EM/PM's are hired, they take over that role for the new Group.

Moving Stable Counterparts between stages

At times it may be necessary to transfer a stable-counterpart from one team to another. In cases where this team member's previous role will be backfilled, follow the Department Transfer Process. In cases where the role will not be backfilled (i.e. the role was shifted from one team to another), the following steps should be taken to ensure leaders in the relevant stages are informed and can help guide their teams through the changes in team member allocation:

1. Create an issue in a private project or a Google Doc with limited access using the following team-realignment issue template, which states the purpose of the reallocation and helps define a communication plan notifying teams affected, the leaders impacted, and the rest of the organization. Key details to cover:
 1. List out Who, what and why
 1. Identify DRIs for action items related to the team changes
 1. Define a transparent communication plan to execute against and assign tasks, as part of the communication timeline
 1. Assign the issue to those who are DRIs or have tasks related to the realignment
 1. Once the issue has gotten approval from leadership and impacted parties have been made aware of the reallocation of team members, move the issue to a public repository (such as Product, [www-gitlab-com](https://www.gitlab.com), or a project specific to your team) in accordance with our Transparency value, making these changes transparent to all team members.

When a Product Manager inherits an existing category from another product manager

In addition to making the appropriate changes listed above to ensure that the category is reflected as part of the right group, stage and section, the following steps are also important.

1. The PM doing the hand-off prepares an issue detailing process for change using the template.
1. The PM receiving the category initiates a review of the direction page and asks the receiving team EM and designer to review the page as well. This helps the team gain basic understanding immediately.
1. The PM's together identify DRIs and clarify who owns the communication (It is preferred that questions to help understand the category go to the PM instead of the rest of the quad team members. This allows for the rest of the quad team members to stay focus on their new responsibilities).
1. Create a recording for broader distribution

Rapid Action

Rapid Action is a process we follow when a critical situation arises needing immediate attention from various stakeholders.

What deserves Rapid Action

Any problem with both high severity and broad impact is a potential Rapid Action. For example, a performance problem that causes latency to spike by 500%, or a security problem that risks exposing customer data.

What if it only affects one customer?

If the problem only affects one customer, consider a customer escalation process as the alternative.

Rapid Action Process

When a situation is identified as a potential Rapid Action the following actions are recommended:

1. Identify the problem(s) to solve. Ensuring we have a data driven approach ensures we have a measurable way to quantify and track those metrics throughout the rapid action.
1. Identify the exit criteria or goals that would resolve the stated problems of the rapid action. Highlight key dashboards or charts the DRI should be tracking to determine whether progress is trending in the right direction, so that adjustments to the work, goals, or allocation of individuals to the rapid action initiative can be made as needed. Ideally we are able to define and agree to the exit criteria amongst stakeholders, prior to fully dedicating engineers to execute on the effort.
1. Continue to iterate on the above step until the stated goals are achieved, or have the DRI continue to iterate on the goals of the rapid action. The DRI will be the one to work with stakeholders to discuss tradeoffs between progress made in the stated effort vs. the time commitment of Engineering DRIs.

Administrative Tasks

1. Create an epic in the GitLab-org group (This will link you to the epic creation page) group describing the problem and the resolution criteria as briefly and precisely as possible.
 1. Apply the rapid action label.
 1. If the problem is related to security, make the epic confidential.
 1. If possible, list existing issues that are in the scope of this Rapid Action.
1. Identify the stakeholders involved and @-mention them on the epic. It is a good idea to over-communicate this problem, both for raising awareness and gathering ideas.
1. Decide on a Directly Responsible Individual (DRI) and clearly note this in the epic description. This decision should be made as soon as possible, so leadership and responsibility is clear. However, because this decision must be made quickly, it should not be considered a final decision.
1. Clear the schedules of the DRI and anyone else who is expected to be involved in resolving the problem. Rapid Actions are both important and urgent, so they should displace less important and urgent work. For example, if an engineer is asked to help resolve a Rapid Action, the deliverables currently assigned to them should be re-assigned or re-scheduled. Rapid Actions are stressful and time-consuming, so quickly shifting other work is a way to soften the impact.
1. Set up a daily business day standup and invite the correct stakeholders and participants.

Optionally, to facilitate communication, you might:

1. Share a Zoom link dedicated to an immediate discussion.
1. Create a Slack room dedicated to ongoing discussion with the naming convention RA-epic-id

Responsibilities of the DRI

The DRI is responsible for coordinating the effort to resolve the problem, from start to finish. In particular, the DRI is responsible for:

- Maintaining the problem and resolution criteria in the epic description.
- Running the daily standup related to the epic.
- Tasking sub-DRIs as necessary. These sub-DRIs might be responsible for specific parts of the problem (part A/B/C), specific perspectives (Engineering/Infrastructure/Product), specific timezones (AMER/EUR/APAC), etc.
- Ensuring that progress is being made toward mitigation and resolution. This may include

coordinating problem-solving efforts across timezones so we can have GitLab team members working on the problem 24 hours a day.

- Ensuring that work that is in-scope as part of a Rapid Action isn't included in the Development Escalation Processes (a.k.a. infradev).
- Updating stakeholders at least daily.
- Detailing recommended follow-up actions once the problem has been solved (enhancements, refactoring, etc).

Please note that customers can be stakeholders. The DRI can seek assistance with customer communication in Slack at supportescalations.

Status Update Template

The DRI should post a summary status update on the epic at least daily. The following format is recommended (provided here in Markdown format for easy copy/pasting into issues):

markdown

YYYY-MM-DD Update

Progress since last update:

This section describes what changes have been deployed to production and any other notable progress or accomplishments.

Progress expected by next update:

This section describes what you expect to accomplish prior to the next update. For example what work is currently in progress (include links to MRs), when do you expect these to be deployed, what do you expect to be the effect(s)?

Blockers:

This section describes any specific obstacles preventing progress. What is needed to overcome them? Are there team members (e.g. executives, domain experts) these concerns should be escalated to?

Praise:

This section is used to highlight specific praise for team members contributing to the Rapid Action. It is important to say thanks.

Resolution

Once the resolution criteria have been satisfied:

1. Close the epic.

1. Host a retrospective to understand what about the rapid action process could be improved. (Note: there could also be other retros that happen related to more specific sub-efforts of the rapid action, this retro should act as a touch point to ensure collaboration + communication

worked.)

1. Consider making the epic public. If the problem is related to security, ask @gitlab-com/gl-security/secops to determine when the epic can be made public.

1. Communicate the resolution to stakeholders.

1. Consider awarding discretionary bonuses to the people who stepped in to help resolve the problem.

Borrow

Borrow is used when team members are shifted from one team to another temporarily. Team members complete assignment when the work is done. Borrows are meant to align management structures into a single group for coordination and logistics of the effort. It is important to define the work upfront so that it is bounded. We prefer borrows to be for a milestone, but generally can extend to multiple milestones. Any borrow more than 3 months should be reviewed carefully for scope and minimized to the extent possible. Where the ask likely extends beyond 6 months, a realignment should generally be used instead.

It is recommended that quad counterparts are able to seek alignment on their own first for fulfilling a borrow. As an example, in a stage where the quad leadership is already aligned on the most important business priorities, we are able to move quickly by aligning at a lower level and using a lighter version of the Borrow template to document the decision. This example might also make sense at a product group level if multiple groups are able to align themselves, or when multiple stakeholders are required (e.g. different reporting structures) but agreed. This template can also be used for individual team members who pick up work outside their assigned group. For efficiency, this approach should be preferred whenever possible.

If this is not possible or for broader support (e.g. looking far outside your area), EMS, PMs, and Product Design Managers utilize the issue template for Borrow requests. Following this template helps ensure the process is efficient, well-organized, and receives the proper approvals. Identification of the group/individuals that will be borrowed should be done in a private google doc or limited access project. Once a borrow request is approved, details about the impacted groups/team should be added to the issue to communicate the change more broadly.

Every borrow must have a specific deliverable commitment described. The intent is to set expectations and provide team members an opportunity to clearly understand the expectation. Selected team member preference is considered and providing this would also be an opportunity for other interested team members with the required skills to become involved.

When Requiring Specialty

In cases where there is an architectural and/or significant technical project needed to be undertaken, team members with specific skill sets may be required to achieve the goal. In these instances we should endeavor to have a clear goal, clearly defined skill sets, and number of team members needed.

In scenarios where urgency is required, it may be necessary to select specific team members. The impact this has on team members, teams, and the broader organization should also be addressed. All team members with the desired skills should be included in consideration to ensure opportunities are equally available. Meanwhile, try to avoid cascading impacts where possible so that impacted teams can still operate reasonably (e.g., the teams who are giving

shall have at least 3 engineers remaining and a maximum of 1 engineer can be given).

When borrow requests aren't appropriate

We've learned that there are some circumstances where borrow requests aren't appropriate:

- Borrow requests shouldn't be scoped to completing sustaining or forced-prioritization activities when other scheduled work is already happening. Sustaining and forced-prioritization issues are required activities of a Product Group. As a result borrow requests should be framed as a decision to complete required work in addition. For example, borrow requests should not be utilized to resolve security or infradev issues while the product group works on features.

How to have a successful borrow request

We have learned that there is a way to frame a borrow that leads to a more constructive conversation:

- Borrow requests should be scoped to least prioritized work for a given group. This is counter-intuitive, but the argument should be for the weakest work so when it's compared against other team initiatives it's fairly evaluated for the tradeoff. The reasoning here is that if team members within the team could be moved to higher priority work that's the first place to borrow from.
- Borrow requests should not be initiated to deliver an OKR, but OKRs can be referenced in communicating the business need to propose a borrow request. The borrow request should be framed in terms of prioritized work. An OKR goal that is for fixing security issues would be categorized a priority 1 based on our current work.

Active borrow requests

This board contains all proposed and active borrow requests.

Scope reassignment

A scope reassignment is used when the features or effort of work will take longer than 6 months and less than a year, but it is not necessary to realign teams permanently in order to deliver the work. Rather, the team reporting structures are maintained and the product groups are directed to work on the product scope of another group, stage, or section.

It is recommended that with a scope reassignment, the product groups align on what portions of scope will be delivered by each group. This is done with epics and issues, as well group:: labels.

How are scope reassignments, borrows, and realignments different?

A scope reassignment is different from a borrow and realignment because team structure is not changing, instead the items the product groups are working on change. Additionally, borrows are to be used temporarily (less than six months) and realignments are permanent. In the case of a scope reassignment, a product group would maintain its long term charter and vision/investment level, although for up to one year will work on another product area's scope. The reason for a scope reassignment is to reduce the toil of changing managers and team structures while still

activating on strategic efforts requiring more headcount.

GitLab.com In App messages (Broadcast Messaging)

Broadcast Messaging is a great tool for acquiring user feedback from within the product. This tool allows for general, one-time, important announcements or for users to be recruited during or after interacting with specific workflows within the product. Currently, broadcast messaging can be targeted by URL, and user information can be passed in order to personalize the message as well as the response.

There are two types of Broadcast Messages - banners and notifications. Currently banner notifications also send messages in Git responses, but this is under review.

How to use Broadcast Messaging

All broadcast messaging efforts must follow all guidelines in order to be deployed to GitLab.com. Create an issue in the GitLab.com/Product project using the Broadcast Messaging template and assign it to @justinfarris & @ampesta for review. The In App Messaging board is used to prioritize all messages in queue and in flight.

See issue template for usage guidelines. If the message requires a group to do work (for a banner message for instance) you may want to create an issue in the gitlab/gitlab-org project for better visibility.

SECTION: product/product-processes/pm-tips-and-tricks/_index.md

Overview

This section of the handbook is a collection of product management processes that can be leveraged in your practice as product management. Some of these are best practices and suggestions that are not required in your day to day as product manager, while others are highly recommended workflows that are tried and true paths leading to successful results. These are sourced by our Product Management Department and regularly reviewed by our Director+ Product Management leaders.

Tips and Tricks

Internal and external evangelization

Before shipping a new or updated feature, you are responsible for championing it, both internally and externally. When something is released, the following teams need to be aware of it as they will all need to do something about it:

- Marketing: depending on the importance of the feature, we need the help of marketing to promote this feature on our different communication channels.
- Sales: sales needs to know what's new or changed in the product so they can

have better arguments to convince new or existing customers during their sales process.

- Support: as they are in constant contact with our users and customers, support should know exactly how our products work.

You can promote your work in several ways:

- start with documenting what will be released and share this documentation with the different teams
- schedule meetings, if you think it's important, with the teams listed above.

When referencing issues in written communication using just the issue number 123456 and a link is not low-context communication. Instead use the title of the issue and the link or the issue number and description of the problem that issue will solve:

- Good: We will next be working on Detect and display code coverage reports on MR. OR We will next be working on gitlab21549 which will help developers view code coverage reports directly in GitLab instead of losing context by looking in another tool while reviewing an MR.
- Avoid: We will next be working on 21549..

In order to support findability and to clearly articulate when we change our minds especially when it comes to product direction, category changes, shifts in investment themes, or priorities for engineering, Product Managers must evangelize these changes in multi-modal communication channels to ensure our users and customers aware.

Some internal methods for communication include:

- Sharing the updates various product-based Slack channels such as: product, s, g, or f Slack channels
- Cross-posting changes in direction or categories into customer-success and if they impact use cases tag @cs-leadership for awareness
- Recording a quick video and sharing with Customer Success that discusses direction updates. Use sync meetings as needed to facilitate efficient communication.
- Collaborate with the Field Communications team to determine if a larger internal communications plan/approach is necessary for the Field (Sales, Customer Success, Channel & Alliances) team.
- Aggregating and sharing highlights of monthly direction page updates at the Section-level across the organization

External channels for consideration linking direction pages to:

- Twitter, LinkedIn, or other social accounts
- Sharing outreach emails via account teams
- Recording walkthroughs on Unfiltered and promoting on social accounts
- Writing a blog about the changes, if they are significant or disruptive

Writing to inspire action

As a PM, it is important to remember a bias towards action (and other value actions like sense of urgency, make a proposal, boring solutions, write things down, don't wait, make two way

doors decisions)

which enables PMs to drive an async discussion to being action oriented. Every time you write a comment or create an issue ask yourself: Will this allow us to take an action and move us forward?

Writing about features

As PMs we need to constantly write about the features and upgrades we ship: in a blog post, internally to promote something, and in emails sent to customers. There are some guidelines that one should take into account when writing about features, the most important being a clear communication of the problem we're solving for users.

When writing about a feature, make sure to cover these messaging guidelines which help produce clear internal and external messaging. Please also keep in mind that we should avoid using acronyms that others may not recognize, such as "MVC" for Minimal Valuable Change. For more guidance you can visit our writing style guidelines.

Let's highlight the messaging guidelines mentioned above with a concrete example, Preventing Secrets in your repositories, that we shipped in 8.12.

- Start with the context. Explain what the current situation is without the feature. Describe the pain points and connect back to our Value Drivers (in this case Reduce Security and Compliance Risk).

- > It's a bad idea to commit secrets (such as keys and certificates) to your repositories: they'll be cloned to the machines of anyone that has access to the repository. If just a single one is insecure, the information will be compromised. Unfortunately, it can happen quite easily. You write `git commit -am 'quickfix' && git push` and suddenly you've committed files that were meant to stay local!

- Explain what we've shipped to fix this problem.

- > GitLab now has a new push rule that will prevent commits with secrets from entering the repository.

- Describe how to use the feature in simple terms.

- > Just check the checkbox in the repository settings, under push rules and GitLab will prevent common unsafe files such as `.pem` and `.key` from being committed.

- Point to the documentation and any other relevant links (previous posts, etc).

Here are some additional examples of well written release blog posts for inspiration:

- Issue Board Work In Progress Limits
- Parent-Child Pipelines
- Drag-and-drop Design badges

- Render charts in GitLab issues using a Grafana URL

Recording videos to showcase features

In addition to the written medium, video is an important medium that caters to the different goals you are trying to accomplish and learning styles of your audience. Depending on the type of video you are recording, there are some guidelines to keep in mind.

As our documentation guidelines actively encourage linking video content, please consider following the Documentation Style Guide section on language, and working with your technical writing team to include links to your speed runs, walk-throughs and demos at relevant locations in the product documentation.

Using GIFs

Animated GIFs are an awesome way of showing of features that need a little more than just an image, either for marketing purposes or explaining a feature in more detail. Checkout our guide to Making Gifs!

Speed Run

Speed runs are informal videos meant to focus on a single workflow and the experience for performing that workflow. It should not require much planning and is typically short in duration (less than 5 min.). This video type is meant to inform and not necessarily to influence buyers.

Examples:

- GitLab Unfiltered Speed Runs Playlist
- Remove docker images via CI/CD speed run

Demo

Demos are scripted recordings meant to influence buyers. Generally has higher production value and typically involves both a slide-style presentation and/or live screen-sharing. Duration varies depending on the topics being covered.

Examples:

- GitLab for the Enterprise Demo

Walk-through

Product walk-throughs are informal videos meant primarily for an internal audience as a recorded, visual form of product critique. Walk-throughs typically focus on the user experience across categories and workflows within a Product Manager's product scope. There are particular benefits to walk-throughs which span product hierarchy boundaries (multi-category, multi-stage, multi-section) as they help highlight disjointed experiences across our single-application.

Walk-throughs are typically longer in length as they cover more ground and often involve some

"live" troubleshooting and are best performed with no planning. Use the Product walk-through issue template when creating a walk-through.

Examples:

- Auto DevOps setup and usage walk-through

SECTION: product/product-processes/product-okrs/_index.md

SECTION: product/ux/_index.md

Hello

We're the GitLab User Experience (UX) department. We comprise four areas to support designing and building the GitLab product.

- UX Research
- Technical Writing
- Product Design
- Foundations - Pajamas

Our goal is to make our product easy to use, supportive of contributions from the wider GitLab community, and built for a diverse global community. We want GitLab to be the easiest and most delightful product in its class.

How we work

- We support all users from beginners to experts. We believe that GitLab software should be unintimidating and accessible for a beginner, without oversimplifying important features for advanced users. We stay with users every step of the way to help them learn fast as a beginner and then become an expert over time.
- We're building one product, together. We're highly focused on ensuring that no matter how big our product gets, the entire experience stays cohesive, consistent, and interconnected.
- We're humble facilitators of user experience design. Everyone is a designer; everyone can contribute. We are not egotistical, moody experts who alone hold the keys to user delight. We encourage Product Managers, Engineers, and the wider GitLab community to contribute to creating an exceptional user experience.
- We look for small changes and big impacts. Sometimes the simplest, most boring solution is what is needed to make users successful. We want our UI to stay out of the user's way. We work iteratively to make modest but valuable changes that make users more productive, faster, and better at accomplishing their tasks.
- We're informed by empathy. We're human, and we design for humans, so we strive for understanding, self-awareness, and connection. We are quirky, and we introduce our quirks into designs when appropriate.

- When we find problems that are simple to fix, we are empowered to make those changes ourselves. If a change will take you less than 15 minutes to make (for example, a minor change to our website or microcopy in the product), then start with an MR instead of an issue. By making the change yourself, you are taking immediate action to improve our product, and you might learn a new skill, too! If it seems simple, but you have questions, remember that there are people who can help you with code changes both in the UX department and across the company.

We work closely with the community, and our stable counterparts Product Managers (PM), Frontend engineers (FE), Backend engineers (BE), Quality engineers, and the Brand team. We follow GitLab's shared process referred to as the Product Development Flow and R&D Interlock.

- PMs define the "what" and "why" to lead the product direction. These are the benefits we provide to users. It's informed by gathering customer and user feedback in partnership with UX Research.
- Product Designers define "how" the direction is experienced. It's how users interact with the product to gain the benefits.
- Engineers define "how" the product is built to meet the product and experience direction from PM and Product Design.

Workflows

- UX Researcher
- Technical Writing
- Product Designer
- Product Design Manager
- Cross-functional Prioritization

Headcount planning

Every Product Designer is aligned with a PM and is responsible for the same customer benefits the PM oversees. Technical Writers each support multiple stage groups. UX Researchers support multiple groups within a section.

In the spirit of having stable counterparts, we plan headcount as follows:

- One Product Designer for every stage group.
 - 1:1 ratio of Product Designers to PM (excludes stage groups with no user-facing impact or, in some cases, stage groups with low usage).
 - 1 Product Designer for 1-3 Frontend engineers; 2 Product Designers for 4-5 Frontend engineers.
- One Technical Writer for up to three stage groups.
 - 1:3 ratio of Technical Writers to stage groups.
 - Approximately a 1:21 ratio of Technical Writers to Engineers.
- One UX Researcher for up to 5 stage groups.
 - 1:5 ratio of UX Researchers to Product Managers.
 - Approximately a 1:35 ratio of UX Researchers to Engineers.
- Manager support that's appropriate for the function.
 - Approximately a 1:5 ratio of Managers to direct reports for UX Research and Product Design.
 - Approximately a 1:7 ratio of Managers to direct reports for Technical Writing.

UX labels

GitLab uses labels to categorize, prioritize, and track work. The following is a breakdown of the labels most directly related to the UX workflow. An overview of all the label types and uses can be found in the contributing doc.

- UX label: Indicates that UX work is required on this issue. These issues can be new features, ideas for improvement or anything else where UX should contribute their expertise.
- Inclusion label: A change to GitLab that promotes inclusion as it relates to our diversity value.
- Inclusive design label: Considering, exploring, and evaluating the different ways someone would access, interact with, or contribute to content that results in a more accessible experience.
- Accessibility and scoped accessibility labels are used to identify issues with accessibility impact. The scoped labels should be added after an accessibility audit has validated the impact and used in combination with priority and severity labels to triage an issue.
 - Accessibility label: Issues that contain actionable items that help create an accessible product experience.
 - Accessibility-audit label: Issues related to auditing existing experiences in order to understand possible accessibility-related improvements.
 - Accessibility-ops label: Issues related to building accessibility into our internal workflows.
 - accessibility::critical: Prevents some or all users from performing critical tasks with no possible workaround.
 - accessibility::high: Prevents some users from performing critical tasks. A workaround may exist, but not without creating frustration and inefficiency.
 - accessibility::medium: Prevents some users from performing non-critical tasks, or where the user experience is seriously degraded for users with certain assistive technologies.
 - accessibility::low: The user experience is degraded for users with certain disabilities or using certain assistive technologies, but users can still accomplish tasks.
- learnability label: Issues that address learnability problems by helping users quickly become familiar with GitLab features.
- Scoped workflow labels from the Product Development Flow should be used to indicate where an issue is in the development lifecycle. Issues can move between workflow labels as many times as necessary, and not all labels will be applicable to every issue. Issues that require UX would use one of these labels as defined in the Product Development Flow:
 - workflow::validation backlog
 - workflow::problem validation
 - workflow::design
 - workflow::solution validation
- Pajamas component lifecycle labels are scoped labels used for creating and updating Pajamas components. Label usage guidelines can be found in the Pajamas component lifecycle documentation.
- UX problem validation label: Indicates that the issue requires UX work to validate that the problem is relevant to users. We use this label in addition to the Product Development Flow scoped labels, so that we can track validation efforts over time in our UX Performance Indicators.
- UX solution validation label: Indicates that the issue requires tasks to validate that the proposed solution is technically feasible and meets user needs. We use this label in addition to the Product Development Flow scoped labels, so that we can track validation efforts over

time in our UX Performance Indicators.

- UI polish label: Indicates the issue covers only visual improvement(s) to the existing user interface.
- Deferred UX label: Deferred UX results from the intentional decision to deviate from the UX vision or MVC, which sacrifices the user experience. Deferred UX labeled issues are to be included in subsequent releases. Use this label to indicate that the UX released does not meet:
 - UX and Pajamas specifications
 - Usability standards
 - Feature viability standards

This label is applied to any follow-up issues that address a UX gap. It does not apply to the issue or merge request that created the Deferred UX. For example, if the agreed MVC design solution is not fully realized due to release pressures or implementation oversight, that's considered Deferred UX.

If the design is implemented correctly but unforeseen UX issues are identified, it is not considered Deferred UX.

If in doubt about when to apply this label, use the following rule: If you can say "This UX problem did not originate from an issue or merge request," then it's just UX, not Deferred UX. In case your team makes the decision ship an MVC that contains Deferred UX, it is recommended to create an issue to track it as soon as the change has been released.

- Learn more about Deferred UX as a UX Department Performance Indicator.
- UX Paper Cuts label: Used to prioritize and track work from the UX Paper Cuts team.
- Seeking community contributions
- System Usability Scale (SU)